

Can Deterministic Network Verification Reduce Undetected Faults in LLM-Generated Configurations?

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory study (study id c1-gnr-vv-2026-0001) to measure whether inserting a deterministic network verifier between an LLM intent→configuration generator and an emulated execution reduces the proportion of configurations that would execute with operational faults. Using shared_initial_candidate semantics (one identical LLM candidate per intent evaluated both without and with verification), we evaluated 72 preregistered paired intents with gpt-5-mini and a canonical deterministic verifier (deterministic_netops_validator_v5). Execution completed with 144 episodes (72 pairs) and 72 model calls. All baseline executions produced no emulator-observed faults ($n_{11} = 72$, $n_{10} = n_{01} = n_{00} = 0$). The paired difference in undetected-fault proportion is 0.0 (paired bootstrap 95% CI [0.0, 0.0], exact McNemar two-sided $p = 1.0$; bootstrap seed = 1729, resamples = 10000). Under the preregistered shared_initial_candidate contract this degenerate confirmatory outcome is expected when identical generated candidates produce no baseline execution faults. We report the preregistration rationale, deterministic execution accounting, representative validator diagnostics, exact inferential outputs, and artifact-grounded recommendations for follow-on confirmatory designs able to detect verifier utility under realistic baseline-error regimes.

I. INTRODUCTION

Generative large language models (LLMs) are increasingly proposed to translate high-level operator intents into device-level network configurations. This intent→configuration automation can speed change pipelines but risks silently violating reachability, access-control, ordering, or workflow invariants and thereby causing outages if generated text is executed without additional checks. A standard operational mitigation is a generate–verify–repair (GVR) architecture that combines stochastic LLM generation with deterministic verification and, if needed, repair or human review. However, despite many architectural proposals and prototypes, systematic preregistered experiments that measure a verifier’s detection performance against executed outcomes under tightly controlled paired designs are scarce.

This paper answers a narrowly scoped, preregistered research question: does inserting a deterministic network verifier between an LLM generator and emulator execution materially reduce the proportion of configurations that execute with operational faults when the identical generated candidate is evaluated both without and with verification? That estimand isolates verifier detection from improvements that could arise from re-sampling, prompt-engineering, or repair-enabled iterations. We executed study c1-gnr-vv-2026-0001

using gpt-5-mini and deterministic_netops_validator_v5 across 72 preregistered paired intents under shared_initial_candidate semantics. The run completed with no technical failures, produced a degenerate confirmatory outcome (no baseline emulator faults), and yields a paired difference of 0.0 (95% CI [0.0,0.0], exact McNemar $p = 1.0$). Rather than treating this as a null failure, we analyze why it occurred given the preregistered contract and archived artifacts, and we provide concrete, artifact-grounded guidance for designs that can detect verifier utility when baseline error prevalence is non-negligible.

II. RELATED WORK

Work on LLM-driven NetOps spans intent→configuration translation, multi-agent/tool-augmented orchestration, and architectural proposals for combining stochastic generation with deterministic checks. Prior empirical studies demonstrate that modern LLMs can produce device-level configurations from high-level intent in constrained vendor-dialect settings and curated evaluation suites; these results show promise but are typically scoped to limited task families and curated datasets rather than broad production traces [1]. That line of work motivates leveraging LLMs to reduce operator effort while highlighting the need for safety controls when configuration semantics and ordering matter.

Complementing single-agent generation work, multi-agent and tool-augmented frameworks have been proposed that explicitly integrate verifiers, tool calls, or specialized modules into LLM-driven NetOps pipelines to reduce regressions and coordinate multi-step changes [2]. These systems illustrate practical architectures for inserting deterministic checks or domain services into agentic workflows, and their evaluations commonly rely on emulators, curated scenarios, or prototype deployments. Such prototypes suggest feasible integration points for verifiers but do not, in the supplied records, provide preregistered paired measurements of verifier detection performance against executed faults.

A related and more general strand of work articulates and evaluates generate–verify–repair (GVR) patterns outside NetOps—particularly in code- and software-repair domains—where stochastic generators produce candidates that deterministic checkers validate and, when necessary, trigger repair iterations [3]. These studies document how deterministic validation can convert nondeterministic generation into a correctness-convergent workflow; they also show that the empirical util-

ity of verification depends strongly on the baseline error rate of generator outputs and on whether the architecture permits verifier-triggered repairs or resampling. Translating these insights into NetOps requires coupling LLM outputs to domain-appropriate verifiers and measuring detection relative to executed outcomes.

Canonical deterministic network-verification techniques—exemplified by header-space analysis and related formal reachability/policy analyzers—provide precise, semantics-aware invariants for reachability and ACL-style policy checks and are natural candidates for the ‘verify’ stage in GVR architectures [4]. Such verifiers can deterministically flag violations of specified invariants or workflow constraints expressed over packet headers, interface states, and rule orderings. However, the literature lacks, among the supplied records, preregistered paired experiments that measure how often these canonical verifiers would have detected faults that would otherwise manifest when LLM-generated configurations are executed in an emulator or live device.

Taken together, the verified literature establishes three pertinent points: (i) LLMs can produce plausible device-level configs from intent in constrained settings [1]; (ii) system proposals and prototypes show how verifiers and tools can be integrated into multi-agent NetOps workflows [2]; and (iii) GVR patterns can yield empirical correctness improvements in other domains when baseline generator errors are non-negligible and when verifier-triggered repairs or resampling are allowed [3]. What remains underexplored in the supplied evidence is a preregistered, paired measurement that isolates verifier detection on identical candidates—i.e., does a canonical deterministic verifier reduce the probability of an execution fault for the same LLM-generated candidate when the candidate is executed with and without prior verification? Our study directly targets this measurement gap by adopting a shared_initial_candidate paired design and reporting exact execution-accounting artifacts to make that detection question reproducible and auditable.

III. METHODOLOGY

Preregistration and estimand. We preregistered study c1-gnr-vv-2026-0001 and froze the design and analysis prior to observing outcomes. The primary estimand is the paired reduction in undetected operational-fault proportion (paired_success_rate_difference_guarded_minus_baseline). The confirmatory hypothesis H1 targeted an absolute paired reduction of 0.20 with two-sided $\alpha = 0.05$ and power ≥ 0.80 ; a sample-size heuristic returned ≈ 63 pairs and we preregistered $N = 72$ pairs (24 per topology-difficulty stratum) to allow margin for deterministic exclusions.

Shared_initial_candidate semantics and experimental contract. The execution_contract enforced shared_initial_candidate semantics: for each intent exactly one initial LLM generation was produced and that identical candidate was evaluated in two conditions. Baseline: execute the shared candidate in an isolated emulator and record emulator fault/no-fault. Guarded: run the canonical

deterministic verifier (deterministic_netops_validator_v5) on the same candidate; if the verifier flags violations the guarded outcome is recorded as prevented (no execution); if it does not flag, execute the same candidate in the emulator and record the emulation outcome. This paired structure ensures any observed paired difference arises solely from verifier detection preventing an otherwise-executed fault.

Model, adapter, and verifier configuration. The declared model in execution/model_configuration.json is gpt-5-mini (provider = openai_responses, max_output_tokens = 2000, maximum_attempts_per_call = 1). execution/model_configuration.json records temperature = null; this indicates provider-default runtime sampling semantics and is archived as a residual sampling-parameter uncertainty. The adapter family was hosted_netops_gvr_v1. The canonical verifier is deterministic_netops_validator_v5 (validator_version = "5.0") using a full_intent_constraint_validation rule set that outputs per-episode validity verdicts and structured validator_traces (parsed_command_count, required_command_count, validation_before/after states, violation_codes, difficulty metadata). All verifier decisions and traces were archived per episode under execution/scoring/.

Task-bank construction, contamination controls, and stratification. Tasks were deterministically generated by deterministic_netops_task_generator_v5. Each task manifest entry encodes a natural-language intent, a machine-structured required_sequence (ordered field changes), allowed_touched_fields, initial and expected final states, and difficulty labels (medium/hard/very_hard). The preregistration required deterministic exact-match contamination checks against public corpora with explicit exclusion rules. analysis/contamination_summary.csv records zero exact-match exclusions for confirmatory pairs. The confirmatory sample retained the preregistered stratification (24 pairs per stratum).

Execution protocol, retries, and deterministic accounting. The missingness_plan allowed up to two additional retries (three attempts total) for transient hosted-API errors on generation calls and required full logging of retries and provider events. The run started at 2026-08-31T06:56:23.065707Z and completed at 2026-08-31T07:06:28.665518Z (≈ 10.1 minutes wall-clock). The execution manifest reports planned_episode_count = 144, completed_episode_count = 144, failed_episode_count = 0, and model_calls_used = 72 (one shared generation per pair). Provider-call audit (deterministic_reconciliation.json) shows hosted_model_call_event_count = 72, completed_hosted_model_call_event_count = 72, cache_miss_count = 72, and stage_counts = {shared_initial_generation: 72}.

Scoring, validation, and inferential procedures. Each episode’s verifier emits a score and validator_trace and the primary analysis used paired_binary_analysis_v1 to form the 2×2 paired contingency table (guarded $\times$ baseline). The preregistered inferential test is an exact McNemar two-sided test

when discordant pairs exist. We computed paired bootstrap-percentile 95% confidence intervals for the paired difference using 10,000 resamples with `bootstrap_seed = 1729` (analysis/analysis_log.jsonl). Pre-specified subgroup confirmatory comparisons (topology complexity and policy type) were registered with Holm correction; these are archived but become non-informative in the absence of discordant pairs. All analysis was executed deterministically by the registered analysis executor and archived under analysis/.

Additional methodological notes (artifact-grounded). The validator’s `full_intent_constraint_validation` rule set enforces sequence and field constraints encoded in the task manifests; validator traces include `parsed_command_count` and `required_command_count` and a `difficulty.level` tag (examples below). Per-episode JSON scoring artifacts under `execution/scoring/` record `normalized_configuration`, `validation_before/after` final_state snapshots, and `violation_codes` arrays that were systematically inspected during analysis.

IV. RESULTS

Execution and manifest summary. The autonomous pipeline completed without technical failures and with full deterministic accounting. Key run-level figures from the archived manifests are: `planned_episode_count = 144`; `completed_episode_count = 144`; `failed_episode_count = 0`; `model_calls_used = 72`; `artifact_count = 510`. The run-level timestamps (`started_at_utc = 2026-08-31T06:56:23.065707+00:00`; `completed_at_utc = 2026-08-31T07:06:28.665518+00:00`) indicate a wall-clock duration of ≈ 10.1 minutes. `analysis/condition_summary.csv` (archived) records `baseline_successes = 72`, `baseline_failures = 0` (`success_rate = 1.0`) and `guarded_successes = 72`, `guarded_failures = 0` (`success_rate = 1.0`).

Paired contingency, inferential outputs, and bootstrap diagnostics. Aggregating across the 72 confirmatory pairs produced the exact paired contingency counts `n_11 = 72`, `n_10 = 0`, `n_01 = 0`, `n_00 = 0` (`guarded  $\times$  baseline`). The primary estimand `paired_success_rate_difference_guarded_minus_baseline = 0.0`. The preregistered exact McNemar two-sided test produces `p = 1.0` given the absence of discordant pairs. The paired bootstrap-percentile 95% confidence interval for the paired difference (`bootstrap_seed = 1729`; `bootstrap_resamples = 10000`; analysis/analysis_log.jsonl) is `[0.0, 0.0]`, reflecting the degenerate sampling distribution when all paired outcomes are concordant. `analysis/paired_contingency_table.csv` and `analysis/condition_summary.csv` contain these tabulations and are archived.

Per-episode validator diagnostics and representative exemplars. We inspected archived per-episode scoring JSONs under `execution/scoring/` and representative task-manifest entries under manuscript evidence. Across the sampled exemplars the verifier produced no violation flags. Representative excerpts from archived scoring artifacts: - pair-000001 (task-000001): `difficulty.level = "medium"`; `parsed_command_count = 4`; `required_command_count = 4`; `validator_version = "5.0"`;

`violation_count = 0`; `normalized_configuration` equals the shared candidate (`execution/scoring/task-000001-*.json`). - pair-000002 (task-000002): `difficulty.level = "hard"`; `parsed_command_count = 2`; `required_command_count = 2`; `violation_count = 0`. - pair-000003 (task-000003): `difficulty.level = "hard"`; `parsed_command_count = 6`; `required_command_count = 6`; `violation_count = 0`. For additional archived representative tasks the task manifests encode `required_command_count` and `difficulty` tags (examples: task-000004 `required_command_count = 4`, `difficulty.level = "very_hard"`; task-000005 `required_command_count = 3`, `difficulty.level = "very_hard"`; task-000006 `required_command_count = 6`, `difficulty.level = "very_hard"`). In all archived scoring JSONs for confirmatory pairs the `verifier_traces` report `violation_count = 0` and `validation_after.valid = true`, demonstrating that the verifier processed each shared candidate and did not flag the candidate as violating the manifest-encoded constraints.

Contamination, missingness, and sensitivity analyses. `analysis/contamination_summary.csv` records zero exact-match contamination flags for confirmatory pairs. `analysis/missingness_summary.csv` records `complete_pairs = 72` and zeros for failed or unscorable episodes. The preregistered sensitivity analyses (including the conservative treatment that would count excluded pairs as failures and bootstrap diagnostics) were executed by the registered analysis executor; because there were no exclusions and no discordant pairs these sensitivity analyses reproduce the degenerate numeric conclusion (`paired_difference = 0.0`). All analysis artifacts (`analysis/paired_contingency_table.csv`, `analysis/condition_summary.csv`, analysis/analysis_log.jsonl) are archived and reference the bootstrap seed and resample count used.

Deterministic reconciliation and provider-call audit. The `deterministic_reconciliation.json` confirms accounting consistency: `completed_episode_count = 144`, `complete_pair_count = 72`, `baseline_success_count = 72`, `guarded_success_count = 72`, `n_11 = 72`, `n_10 = 0`, `n_01 = 0`, `n_00 = 0`, and `all_deterministic_consistency_checks_passed = true`. Provider-call audit fields show `hosted_model_call_event_count = 72`, `completed_hosted_model_call_event_count = 72`, `failed_hosted_model_call_event_count = 0`, `cache_miss_count = 72`, and `stage_counts = {shared_initial_generation: 72}`, consistent with the preregistered `shared_initial_candidate` semantics and the execution manifest.

Interpretation of the degenerate outcome (evidence-grounded). The complete absence of baseline emulator faults in the archived confirmatory corpus is the proximate cause of the degenerate paired result. The archived artifacts together document measurable contributors: (1) the deterministic task generator produced constrained intents with manifest-encoded `required_sequence` and `allowed_touched_fields` that align the generation objective tightly with verifier checks; (2) gpt-5-mini (declared in `execution/model_configuration.json`) produced configurations that matched those manifest constraints across the sampled

tasks; and (3) the `deterministic_netops_validator_v5` rule set (`full_intent_constraint_validation`) enforces the same manifest-encoded sequence and field constraints, producing concordant validity judgments. These archived, machine-verifiable facts explain why the verifier could not reduce executed faults under the `shared_initial_candidate` contract: there were no executed faults to detect.

V. DISCUSSION

Confirmatory interpretation and boundary conditions. The preregistered paired design with `shared_initial_candidate` semantics validly measures verifier detection on identical candidates; because the identical generated candidates produced zero emulator faults the verified paired outcome is a ceiling/null result: the verifier could not reduce executed faults that did not occur. The numeric outputs ($n_{11} = 72$; paired difference = 0.0; McNemar $p = 1.0$; bootstrap CI [0.0,0.0]) are direct consequences of the preregistered contract and observed data and therefore correctly reported as a degenerate confirmatory result.

Artifact-grounded causes of the ceiling outcome. Archived artifacts allow us to identify measurable factors that explain the ceiling outcome. Three concrete, artifact-supported points stand out: 1) Task-manifest constraint alignment. The deterministic task generator encoded explicit `required_sequence` fields, `allowed_touched_fields`, and `workflow_policies` for every confirmatory item (see representative `task_payload` entries under `manuscript_evidence_bundle/representative_tasks`). Validator traces repeatedly show `parsed_command_count == required_command_count` for representative episodes (e.g., task-000001: `parsed=4 required=4`; task-000003: `parsed=6 required=6`), indicating the generated candidate matched the structured sequence encoded in the task manifest. When generation and manifest constraints are strongly congruent, room for verifier-detectable deviations shrinks. 2) Model-task congruence in a synthetic, constrained bank. The declared model (`execution/model_configuration.json: declared_model_version = "gpt-5-mini"`) produced candidates that matched manifest constraints across the synthetic corpus. Because `execution/model_configuration.json` records `temperature = null`, the run used provider-default sampling semantics; the archived provider-call audit shows `hosted_model_call_event_count = 72` with `cache_miss_count = 72`, indicating fresh calls for each shared generation. Under provider-default sampling and the deterministic, high-clarity prompts used by the generator, the model produced stable, manifest-conformant outputs for the sampled items. 3) Verifier-manifest semantic overlap. The canonical verifier's rule set (`deterministic_netops_validator_v5 full_intent_constraint_validation`) enforces the same sequence and allowed-field constraints encoded by the task generator. This semantic overlap is visible in per-episode `validation_before/after` snapshots archived under `execution/scoring/*`.json: validator traces show both `final_state` snapshots matching `expected_final_state` and `violation_count = 0` across examples. When verifier checks are essentially restatements of the task manifest constraints, and generators

respect those constraints, the verifier has limited opportunity to detect faults.

Statistical and design implications. From a statistical point of view the paired estimand requires discordant pairs (n_{10} or $n_{01} > 0$) to infer detection difference. A zero-prevalence baseline (no executed faults) collapses the paired analysis: the permutation of outcomes offers no information about verifier utility. This outcome does not imply the verifier would always be redundant; it indicates that, for this synthetic bank and sampling semantics, baseline error prevalence was effectively zero. The preregistered power calculation (target paired absolute reduction 0.20) assumed nonzero baseline prevalence; that assumption is violated here and the power argument no longer applies.

What these artifacts imply for follow-on confirmatory designs. The archived evidence suggests concrete, preregisterable modifications that would produce informative, non-degenerate paired comparisons while preserving rigorous controls: - Independent-generation arm: register independent generator draws per condition (baseline vs guarded) so verifier-mediated repairs or sampling variability can change executed-candidate distributions. This removes the `shared_initial_candidate` constraint and allows measuring downstream benefits of verification+repair or alternative sampling strategies. - Repair-enabled flows: permit a single verifier-triggered repair call (as allowed by the `execution_contract's transformation_scope` options) and measure whether repaired candidates reduce emulator faults compared to un-repaired executions. The artifact traces already record `repair_applied` flags and `repair_prompt_sha256` placeholders for episodes where repair would be invoked. - Adversarial or ambiguity-seeded task bank: include items with intentionally underspecified intents, vendor-edge cases, or ordering subtleties where `required_sequence` is ambiguous. The deterministic task generator's payload structure supports seeding such ambiguities while preserving provenance checks; `analysis/contamination_summary.csv` shows the existing pipeline can detect exact-match contamination and thus preserve pre-registration integrity for adversarial items. - Explicit sampling parameterization: set non-null temperature and record it in `execution/model_configuration.json` to quantify sampling-driven variability. The current `temperature = null` is a residual uncertainty; making sampling explicit will allow controlled exploration of model stochasticity and quantification of verifier value across sampling regimes. - Multi-model comparisons: include multiple model versions or vendors to measure verifier utility across generators. The `deterministic_reconciliation.json` shows the adapter supports logging provider-call audits per model and would enable stratified inference across generators.

Operational tradeoffs and measurable verifier metrics. The artifact vocabulary supports more granular operational metrics than a binary undetected-fault indicator: verifier true-positive rate (proportion of emulator failures flagged), verifier false-positive rate (proportion of verifier flags that would not produce emulator failures), blocking cost (proportion of verifier flags that prevent benign changes), and repair success rate (proportion of repaired candidates that execute with-

out faults). `analysis/paired_contingency_table.csv` and `analysis/condition_summary.csv` illustrate how a richer contingency scheme can be recorded. Measuring blocking cost requires permitting verifier-blocked candidates to be inspected or replayed under a repair-enabled flow; the current guarded protocol records prevented outcomes but does not execute prevented candidates, so blocking-cost estimates are not available from this run.

Threats to validity revisited with artifact evidence. Internal validity: the preregistered `shared_initial_candidate` semantics intentionally isolates verifier detection but prevents any verifier-influenced change in executed candidate distributions; this choice explains why a degenerate result may occur in low-error baselines. Construct validity: the emulator-observed fault label is a conservative operational proxy, but emulator-level tests omit traffic-dependent and hardware-specific failure modes; validator traces record `final_state` snapshots but cannot capture live-traffic interaction effects. External validity: experiments used a single declared model (`gpt-5-mini`), a synthetic task bank created by `deterministic_netops_task_generator_v5`, and a single deterministic verifier; generalization to other generators, vendor dialects, or production hardware is limited. Conclusion validity: with baseline failure prevalence = 0 the confirmatory paired test lacks the necessary discordance to make inferential claims about verifier usefulness under different baseline regimes.

Practical recommendations grounded in the archive. Based on the archived manifests and traces we recommend that future preregistered studies aiming to quantify verifier utility should: 1) explicitly select or seed a baseline failure prevalence (via adversarial items, ambiguous intents, or historical change examples) so the paired estimand has scope to detect reductions; 2) preregister whether `shared_initial_candidate` semantics or independent-generation semantics will be used, because they answer different operational questions (detection on identical outputs vs. end-to-end improvement under verifier-mediated change); 3) record explicit sampling parameters (non-null temperature) in `execution/model_configuration.json` to remove provider-default ambiguity; 4) include repair-enabled arms to measure both detection and net operational improvement; and 5) log verifier-blocking events and, where ethically permissible, execute or simulate blocked candidates to estimate blocking cost. All these recommendations map directly to fields and artifacts present in the current evidence bundle (task manifests, `execution/model_configuration.json`, `execution/scoring/*.json`, and analysis logs) and can be implemented without inventing new tooling.

Concluding perspective. The degenerate confirmatory outcome reported here is an empirically supported datapoint: under the tested constrained, synthetic conditions the generator and manifest constraints produced valid candidates and a canonical verifier did not change executed outcomes. The key scientific value of the run is therefore methodological and procedural: the archived artifacts demonstrate a fully preregistered, deterministic pipeline for measuring verifier detection under tightly controlled semantics, and they make explicit

which preregistration choices (`shared_initial_candidate` vs independent generation, sampling parameterization, adversarial seeding, repair permissions) determine whether future confirmatory tests can meaningfully quantify verifier utility.

VI. LIMITATIONS

- Confirmatory ceiling/null outcome: the baseline generator produced zero emulator faults on the preregistered task set, preventing direct measurement of verifier detection benefit.
- `Shared_initial_candidate` semantics: the design measures verifier effect on the same generated candidate only and does not assess independent per-condition generation, multi-sample generation, or repairs unless explicitly permitted by the contract.
- Single-model scope: experiments used one declared model (`gpt-5-mini`); results do not imply generality across models or versions.
- Synthetic/emulated tasks: task corpus and emulator executions differ from production devices and live traffic; external validity to production deployments is limited.
- Sampling-parameter uncertainty: `execution/model_configuration.json` shows `temperature = null` (sampling parameters unset); null indicates provider-default runtime sampling semantics and is a residual uncertainty recorded in the archived configuration.
- Residual contamination risk: deterministic provenance and exact-match checks were applied, but private training-data contamination of the hosted model cannot be fully ruled out and remains residual uncertainty.

VII. CONCLUSION

We executed a preregistered paired evaluation (`c1-gnr-vv-2026-0001`) under `shared_initial_candidate` semantics to test whether a canonical deterministic verifier reduces the proportion of LLM-generated configurations that would execute with operational faults. For the chosen model (`gpt-5-mini`), verifier (`deterministic_netops_validator_v5`), and synthetic/emulated task bank, baseline executions produced zero emulator faults and the verifier did not change outcomes (paired difference = 0.0; bootstrap 95% CI [0.0,0.0]; exact McNemar $p = 1.0$). This artifact-grounded null/ceiling outcome is internally consistent with the preregistered design: when identical generated candidates produce no executed faults a verifier cannot reduce executed faults under the `shared_initial_candidate` contract. Measuring verifier utility under realistic error regimes requires preregistered contracts that permit independent or multiple generator samples, repair-enabled flows, adversarial task sets, and multi-model comparisons. The archived artifacts (responses, task manifests, validator traces, execution manifest, and analysis logs) provide a reproducible baseline and a template for those follow-on confirmatory designs and power calculations.

DISCLOSURE STATEMENT

Autonomy and artifacts: This study was executed autonomously after an autonomy lock. Human role:

a Research Director selected the topic family and approved the immutable initial master prompt prior to the autonomy lock; no human manually edited technical manuscript content after the lock. Models and tools: initial generation used gpt-5-mini (declared_model_version recorded in execution/model_configuration.json) orchestrated via the hosted_netops_gvr_v1 adapter; deterministic_netops_validator_v5 (validator_version = "5.0") served as the canonical verifier; an isolated emulator executed candidate configurations. Preregistration: study c1-gnr-vv-2026-0001 was preregistered and frozen before observing outcomes. Immutable master prompt reference: provenance/master_prompt.txt (SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Key archived artifacts include execution/raw_results.jsonl, execution/task_manifest.jsonl, execution/model_configuration.json, analysis/paired_contingency_table.csv, analysis/condition_summary.csv, deterministic_reconciliation.json, and per-episode validator traces under execution/scoring/. The model configuration records temperature = null (provider-default sampling semantics archived in execution/model_configuration.json). Provider-call audit: hosted_model_call_event_count = 72. No human manual manuscript editing or content insertion occurred after the autonomy lock.

REFERENCES

- [1] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313
- [2] Zhaodong Wang, Samuel Lin, Guanqing Yan, Soudeh Ghorbani, Minlan Yu, Jiawei Zhou, Nathan Hu, Lopa Baruah, Sam Peters, Srikanth Kamath, Jian Yang, Ying Zhang, "Intent-Driven Network Management with Multi-Agent LLMs: The Confucius Framework", 2025, 10.1145/3718958.3750537
- [3] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899
- [4] <http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.300.8659>